How to Run a One-Person Hedge Fund on Grok Bot That Finds Profitable Strategies 24/7 on Its Own

A Field Report on Handing Research, Backtesting, Risk, and Execution to a Team of Always-On Agents Running in the Cloud

Abstract*— A solo trading operation has always run into the same limit: the attention of one person. Markets trade around the clock, news lands at any hour, and every credible strategy needs research, testing, risk review, and execution at a pace no individual can hold. Chat-based AI never closed this gap, because a chat model responds once, forgets, and goes idle when the window closes. Grok Bot shifts the unit of work from a conversation to a colleague. Every Bot works on a cloud computer, keeps memory between sessions, runs routines on a schedule, and works alongside other Bots in a shared team. We describe a six-agent desk built on this primitive: a Chief of Staff that coordinates the team, a Scout that tracks news and live activity on X, a Chart Analyst wired to TradingView through its official Model Context Protocol server, a Strategist that turns setups into testable rules, a Risk Officer that holds an ***unconditional veto, and a Trader that sends only approved orders to a broker, starting in paper mode. A strategy counts as profitable here only if it stays positive after costs on data it has never seen, passes a deflated Sharpe test, and holds up across walk-forward windows. Our central argument is that a hedge fund is not an office full of people. It is a process, and a single operator can now run that process autonomously.**

***Index Terms*—Grok Bot, multi-agent systems, autonomous trading agents, solo hedge fund, Model Context Protocol, TradingView, Pine Script, walk-forward testing, deflated Sharpe ratio, fractional Kelly sizing, paper trading, risk veto, scheduled routines.**

I. Introduction: The Attention Gap

Every independent trader runs into the same limit. The market stays open, but the trader has to sleep. An operator who is awake and focused for h hours a day sees only part of a 24-hour market. At $h = 8$:

$$C_{solo} = \frac{h}{24} \approx 0.33\,, \qquad C_{desk} = 1$$

With N actionable signals a day arriving at random hours, the expected number the operator never sees grows with every hour away from the screen:

$$\mathbb{E}[missed] = N\,(1 - C_{solo}) \approx 0.67\,N$$

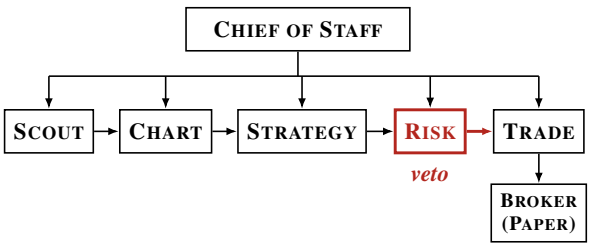
The standard fix has been to automate a single link in the chain: a screener in one place, an alert in another, a script that fires orders. Each tool saves time, yet none of them reasons, and none of them talks to the rest. The operator is still the glue, and the operator's attention is still the upper bound on what the system can do.

This paper takes the operator out of the routine work and leaves the operator in charge of the decisions. What remains is a desk that researches, tests, and trades without pause, and calls on a human only when capital is on the line.

A. Why a Single Chatbot Fails as a Desk

Ask a chat model to trade and it takes the question, returns a confident answer, and goes quiet. Without a schedule, it cannot watch the market. Without persistent memory, it pitches last week's rejected idea as if it were fresh. Without tools, it describes charts it has never opened. Without colleagues, the same model that suggests a trade also signs off on it. Any one of these gaps is fatal. Together they explain why chat-based trading has stayed a demo rather than a desk.

B. Desk Topology



C. A Single Chatbot Versus the Six-Agent Desk

Table I compares one chatbot asked to trade with the same model split across the six-agent desk, on the six functions that decide whether a strategy survives contact with a live market. The gap is not intelligence. It is whether each function has an owner, a schedule, and a written record.

TABLE I

A Single Chatbot vs the Six-Agent Desk

Function	Single Chatbot	Six-Agent Desk
Monitoring	On request only	Scout, every 30 min
Chart analysis	From memory	TradingView MCP
Strategy testing	None	Strategist backtests
Risk control	Self-approved	**Risk Officer veto**
Execution	By hand	Trader, paper first
Memory	Lost each chat	Persistent + journal

D. Roadmap of This Paper

Section II lays out the desk architecture, each agent's routines, and the operating rules all agents share. Section III describes the strategy discovery loop and the statistical gates a strategy must pass. Section IV covers the TradingView and broker integration, including the math behind the custom indicator. Section V treats position sizing, risk limits, and the kill switch. Section VI walks through the paper-to-live promotion ladder, day-to-day operation, and references.

II. Desk Architecture

A. One Bot, One Job

Everything on the desk rests on a single building block. A Grok Bot is an agent with access to a cloud computer, memory that persists between sessions, and routines that fire on a schedule. It logs into tools the way a person would, keeps working whether or not the operator's laptop is on, and can run many routines side by side. Bots message one another, share context, and collaborate in group chats. The design therefore treats every Bot as a hire with one job description, never as a general-purpose assistant.

B. The Decision Path

Write x_t for everything the desk observes at time t: prices, volume, news, and live activity on X. Any action the desk takes is the result of passing that state through the same chain of agents, always in the same order:

$$a_t = T\Big(R\big(S\big(C\big(Sc(x_t)\big)\big)\big)\Big) \tag{1}$$

Here Sc is the Scout, C the Chart Analyst, S the Strategist, R the Risk Officer, and T the Trader. No stage can be skipped. Any stage may return a null action, which ends the path and writes the reason to the journal.

The ordering matters because every stage acts as a filter of its own. If f_i is the chance that stage i lets a bad idea through, and the stages fail independently, the chance that a bad idea is executed is

$$P(\text{bad trade executed}) = \prod_{i=1}^{k} f_i \tag{2}$$

Four filters that each let a bad idea through 30% of the time leave only $0.3^4 = 0.0081$, fewer than one bad trade in a hundred. An agent that both proposes and approves its own trades has no such protection.

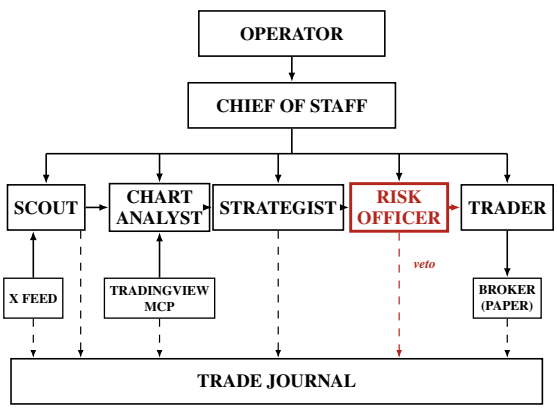


Fig. 1. Six-agent desk architecture and decision path.

C. The Desk Rules

Before acting, every agent reads the same document: the Desk Rules. It sets the markets the desk may trade, the capital, the maximum risk per trade and per day, the cap on open positions, the holding period, and the instruments that are off limits. It also sets two rules of conduct: every claim carries a source and a timestamp, and no agent may make up a price, a figure, or a result. Because all six agents read the same rules, they act as one desk instead of six separate opinions.

D. Routines and the Daily Schedule

Autonomy comes from routines. Each agent owns jobs that fire on a schedule in the cloud, so the desk keeps working without a single prompt from the operator. Table II gives the schedule used here, in US Eastern Time.

TABLE II

Agent Routine Schedule (ET)

Agent	Runs	Produces
Scout	06:30, then every 30 min	Ranked findings
Chart Analyst	08:30, then hourly	Setups scoring 7+
Strategist	21:00 nightly	Tested trade plans
Risk Officer	Every order, 16:15	**Approve or veto**
Trader	Every 15 min in session	Orders and fills
Chief of Staff	07:30, 16:30, Sun 18:00	Briefs and review

E. A Fixed Message Format

Agents talk to each other in exactly one format: TICKER, IDEA, EVIDENCE, NEXT STEP. A single format leaves no room for ambiguity and lets a machine check every message. An idea that shows up without evidence goes back to whoever sent it. An order that reaches the Trader without an **APPROVED** stamp from the Risk Officer is rejected.

F. The Trade Journal

Grok Bot keeps memory across sessions, but the operator cannot read that memory directly. The desk therefore writes every idea, score, plan, risk decision, order, fill, and lesson to an external Trade Journal. The journal is the audit trail, the source of calibration data, and the guard that stops a rejected idea from quietly being tested again.

III. The Strategy Discovery Loop

A. From Finding to Hypothesis

Discovery starts with the Scout, which logs every market event with its source and timestamp. Each night the Strategist turns the day's findings into at least ten hypotheses that can be tested. A hypothesis is accepted only if it states its edge, why that edge should exist, and who is on the losing side when it pays. Ideas that cannot name an edge are dropped before any compute is spent on them.

B. The Setup Score

The Chart Analyst scores each candidate on five components pulled from TradingView: trend alignment, distance to a key level, momentum, relative volume, and the strength of the catalyst. Each component s_j gets a score from 0 to 10, and the components are blended with fixed weights w_j:

$$S = \sum_{j=1}^{5} w_j s_j\,, \qquad \sum_{j=1}^{5} w_j = 1\,, \qquad s_j \in [0,10] \tag{3}$$

Only setups with $S \geq 7$ move on to the Strategist. Writing the weights into the Desk Rules stops the score from sliding toward whatever the agent happens to find convincing that day.

C. Backtest Statistics

Every plan is backtested on no less than three years of history. A gross result means nothing before costs, so each of the n round trips is charged a cost c_{rt} for commission, spread, and slippage:

$$R_{net} = R_{gross} - n \cdot c_{rt} \tag{4}$$

Return per unit of risk is the annualised Sharpe ratio of daily net returns r, and the downside is the maximum drawdown of the equity curve V_t:

$$SR = \frac{\bar{r}}{\sigma_r} \cdot \sqrt{252} \tag{5}$$

$$MDD = \max_t \frac{P_t - V_t}{P_t}, \qquad P_t = \max_{s \le t} V_s \tag{6}$$

D. Walk-Forward Validation

One backtest is a single path through history. The desk instead cuts the data into rolling windows, fits the rules on each training window, and scores them unchanged on the window that follows. Rules are never touched once a test window has been seen. A strategy that works in only one window is filed as luck, not edge.

E. The Multiple-Testing Problem

Ten ideas a night adds up to more than 3,600 a year, and some of them will look superb by pure chance. Across N strategies with no real edge, the best observed Sharpe ratio rises roughly as

$$\mathbb{E}[\max SR] \approx \sigma_{SR} \cdot \sqrt{2 \ln N} \tag{7}$$

Every survivor is therefore judged by the deflated Sharpe ratio, which adjusts for the number of trials, the sample length T, and the skewness γ_3 and kurtosis γ_4 of returns, with SR measured per period rather than annualised:

$$DSR = \Phi\left(\frac{(SR - SR_0)\sqrt{T-1}}{\sqrt{1 - \gamma_3 SR + \frac{\gamma_4 - 1}{4} SR^2}} \right) \tag{8}$$

where SR_0 is the expected best Sharpe under N trials from (7), and Φ is the standard normal distribution function.

F. The Gates

In this paper a strategy is called profitable only if it clears every gate in Table III. Failing even one sends it to the graveyard.

TABLE III

Statistical Gates for a Profitable Strategy

Gate	Threshold	Why
Trade count	$n \ge 30$	Statistical weight
Out-of-sample	$R_{net} > 0$	Profit after costs
Deflated Sharpe	$DSR \ge 0.95$	Corrects for luck
Walk-forward	$\ge$70% windows up	Stability
Latest year	Positive	Today's regime
Parameter plateau	Neighbours within 20%	No lucky setting
Noise test	1-bar delay, 2× slippage	Real-world friction

IV. TradingView and Broker Integration

A. MCP as the Interface

Agents reach the markets through the Model Context Protocol, an open standard that lets an AI system call outside tools. TradingView's official MCP server, currently in public beta, exposes quotes, price history, the screener, fundamentals, news, filings, the economic calendar, standard indicators, watchlists, alerts, and the alert log. It requires an Essential plan or higher, serves delayed data while in beta, and cannot write Pine Script. The desk uses it as its eyes, never as a place to execute.

B. Connecting It

The connection is set up from chat. The operator tells a Bot to add a custom MCP server named `tradingview` at the TradingView MCP endpoint, checks the address, authorises the connection, and approves access from the TradingView account. From then on any agent can attach TradingView to a task, and the Chart Analyst reads every chart through it.

C. The Desk Signal Indicator

Since each Bot can drive a browser, the Chart Analyst can write a custom indicator, paste it into the Pine Editor, and add it to the operator's charts. The indicator used here, Desk Signal, combines four components. Trend comes from a ribbon of exponential moving averages with lengths $n \in \{8, 21, 50, 200\}$:

$$EMA_t = \alpha P_t + (1-\alpha) EMA_{t-1}, \qquad \alpha = \frac{2}{n+1} \tag{9}$$

The trend counts as bullish only when the ribbon is fully stacked, $EMA_8 > EMA_{21} > EMA_{50} > EMA_{200}$, and as bearish when the order is reversed. Momentum comes from the 14-period relative strength index:

$$RSI_t = 100 - \frac{100}{1 + RS_t}, \qquad RS_t = \frac{\bar{G}_{14}}{\bar{L}_{14}} \tag{10}$$

where $\bar{G}_{14}$ and $\bar{L}_{14}$ are the average gain and average loss over the last fourteen bars.

A buy signal needs a stacked bullish ribbon U_t, a close above the previous twenty-bar high, volume of at least 1.5 times its twenty-bar average, and RSI above 50:

$$\begin{aligned} BUY_t = \mathbf{1}\big[U_t \wedge C_t > \max H_{t-20:t-1} \\ \wedge V_t \ge 1.5\,\bar{V}_{20} \wedge RSI_t > 50 \big] \end{aligned} \tag{11}$$

Signals are switched off in disorderly markets. With true range $TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|)$, the filter blocks every signal when

$$ATR_{14,t} > Q_{0.90}\big(ATR_{14} \text{ over the last 100 bars}\big) \tag{12}$$

Listing 1 shows the core of the indicator as the Chart Analyst wrote it.

```
//@version=6
indicator("Desk Signal", overlay=true)
e8    = ta.ema(close, 8)
e21   = ta.ema(close, 21)
e50   = ta.ema(close, 50)
e200  = ta.ema(close, 200)
up    = e8 > e21 and e21 > e50 and e50 > e200
dn    = e8 < e21 and e21 < e50 and e50 < e200
volOK = volume >= 1.5 * ta.sma(volume, 20)
atr   = ta.atr(14)
calm  = atr <= ta.percentile_linear_interpolation(atr, 100, 90)
rsi   = ta.rsi(close, 14)
buy   = up and close > ta.highest(high, 20)[1]
        and volOK and rsi > 50 and calm
sell  = dn and close < ta.lowest(low, 20)[1]
        and volOK and rsi < 50 and calm
plotshape(buy, "BUY", shape.labelup, location.belowbar, text="BUY")
plotshape(sell, "SELL", shape.labeldown, location.abovebar, text="SELL")
alertcondition(buy,  "Desk BUY",  "Desk Signal BUY")
alertcondition(sell, "Desk SELL", "Desk Signal SELL")
```

Listing 1. Desk Signal core logic in Pine Script v6.

D. Broker Connection and Tool Isolation

Orders go through Alpaca's official MCP server, run in paper mode. Order tools are the most dangerous capability on the desk, so access is split by role, as Table IV shows. Only the Trader can reach them.

TABLE IV

Tool Access by Agent

Agent	TradingView	Broker
Scout	News, calendar	None
Chart Analyst	Charts, alerts	None
Strategist	Price history	None
Risk Officer	Read only	Account read
Trader	Alert log	**Orders (paper)**
Chief of Staff	Read only	Account read

V. Position Sizing and Risk

A. Fixed-Fractional Sizing

A strategy that clears every gate can still wipe out an account if it is sized badly. The desk sizes each position so that the loss at the stop equals a fixed fraction f of current equity E. With entry price P_e and stop price P_s, the quantity is

$$q = \frac{f \cdot E}{|P_e - P_s|} \tag{13}$$

At $f = 1\%$ and $E = \$10{,}000$, a trade with a \$2 stop distance buys 50 shares, and the worst planned loss is \$100 whatever the instrument costs.

B. Volatility Scaling

Stops sit at a multiple k of the average true range, so the stop moves further out on its own when the market turns volatile. Substituting into (13):

$$q = \frac{f \cdot E}{k \cdot ATR_{14}} \tag{14}$$

Position size falls as volatility rises, with no judgment call from any agent.

C. Fractional Kelly as a Ceiling

For a strategy with out-of-sample win rate p and payoff ratio b, the average win divided by the average loss, the Kelly fraction is

$$f^* = p - \frac{1-p}{b} \tag{15}$$

Full Kelly is too aggressive when its inputs are estimates, so the desk takes a quarter of it, capped by the Desk Rules:

$$f = \min\left(\frac{f^*}{4}, f_{\max}\right) \tag{16}$$

With $p = 0.55$ and $b = 1.5$, $f^* = 0.25$ and quarter Kelly is 6.25%, yet the 1% cap is the one that binds. Kelly sets a ceiling, never a target, and its inputs come only from out-of-sample trades.

D. Correlation and Portfolio Heat

Two positions in correlated instruments are the same bet taken twice. With position weights w and return covariance matrix Σ, portfolio volatility is

$$\sigma_p = \sqrt{w^\top \Sigma w} \tag{17}$$

The Risk Officer counts any two positions correlated above 0.7 as one bet, and caps total open risk, the portfolio heat H, across all n positions:

$$H = \sum_{i=1}^{n} f_i \le H_{\max} = 5\% \tag{18}$$

E. Event Risk

Scheduled events open gaps that no stop can cover. The Risk Officer blocks new positions ahead of earnings, central bank decisions, and inflation releases, and flags any open position whose holding period runs through one.

F. The Kill Switch

Some failures mean the whole desk has to stop. Table V lists the triggers. When one fires, the Risk Officer cancels open orders, closes positions where the table says so, pauses the Trader, and alerts the operator.

TABLE V

Kill Switch Triggers

Trigger	Threshold	Action
Daily loss	$\ge$2% of equity	**Flatten, pause desk**
Losing streak	3 in a row	Pause Trader
Stale data	No update for 2 bars	Cancel all orders
Slippage	Fill >0.5% off plan	Pause, review
Rule breach	Any agent off the rules	Freeze desk
Strategy drift	Below MC 5th pct	Demote strategy

G. Layered Enforcement

Every limit is enforced three times over, independently: in the Desk Rules every agent reads, in tool isolation that keeps order access away from every agent except the Trader, and in hard limits set inside the broker account itself. A prompt can be argued with. A broker setting cannot.

VI. From Paper to Live Capital

A. The Promotion Ladder

Most automated systems break at one moment: the switch from simulated money to real money. The desk never makes that switch in one jump. Every strategy climbs the five rungs of Table VI, and failing any rung drops it back one.

TABLE VI

The Paper-to-Live Promotion Ladder

Rung	To advance	Capital
1. Paper	2 weeks, every trade logged	Simulated
2. Match	Paper agrees with backtest	Simulated
3. Live tiny	Clean fills, no breaches	10% of plan
4. Approval gate	Operator approves each order	Planned size
5. Full rules	Weeks of clean journal	Planned size

B. The Consistency Test

Rung two checks whether paper results agree with the backtest. With n paper trades of mean return $\bar{r}_p$, and a backtest mean $\bar{r}_b$ with per-trade standard deviation σ_b, the desk computes

$$z = \frac{\bar{r}_p - \bar{r}_b}{\sigma_b / \sqrt{n}} \tag{19}$$

A strategy moves up only when $n \ge 20$ and $z > -2$. A strongly negative z means live conditions are eating the edge the backtest promised, usually through costs, latency, or a change of regime, and the strategy goes back to the Strategist.

C. Daily Operation

For the operator, the day comes down to two messages and a handful of approvals. The morning brief sums up the market, ranks the

setups that survived, and lists every veto with its reason. Live orders arrive as a fixed card: ticker, side, entry, stop, target, size, risk, and rationale, and no reply counts as a no. Every Sunday the Chief of Staff compares live results with the backtests, demotes strategies that are fading, and archives failed ideas in a graveyard so they are never tested again.

D. Limitations

This is not a high-frequency system. Routines run minutes apart, so any edge that fades within seconds is out of reach. TradingView data is delayed during the MCP beta. The operator may not be able to choose the underlying model, and cannot read agent memory directly, which is why the external journal is required. Usage comes out of a weekly allowance shared by every agent. Above all, the statistical gates lower the odds of trading on luck, but they cannot promise future profit, and no architecture can turn a strategy without an edge into one that has it.

VII. Conclusion

A hedge fund was never defined by its offices or its headcount. It is defined by a process: research, testing, risk control, and disciplined execution, running without a break. This paper shows that the process can now be handed to a team of always-on agents, each with one job, one schedule, and one record, while a single operator keeps control of the capital. The edge still has to be found. What changes is that the search no longer stops when the operator does.

References

[1] xAI, "Grok Bot is here," *x.ai/bot*, 2026.

[2] TradingView, "TradingView MCP Server," public beta announcement, 2026.

[3] Alpaca Markets, "Alpaca MCP server," *GitHub repository*.

[4] Model Context Protocol, "Specification," *modelcontextprotocol.io*, 2025.

[5] D. H. Bailey and M. López de Prado, "The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality," *J. Portfolio Management*, vol. 40, no. 5, pp. 94–107, 2014.

[6] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu, "Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance," *Notices of the AMS*, vol. 61, no. 5, pp. 458–471, 2014.

[7] J. L. Kelly, Jr., "A new interpretation of information rate," *Bell System Technical Journal*, vol. 35, no. 4, pp. 917–926, 1956.

[8] W. F. Sharpe, "The Sharpe ratio," *J. Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.

[9] J. W. Wilder, *New Concepts in Technical Trading Systems*. Greensboro, NC, USA: Trend Research, 1978.

[10] R. Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd ed. Hoboken, NJ, USA: Wiley, 2008.

[11] TradingView, "Pine Script v6 reference manual," *TradingView*, 2026.